

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE NAME: Database Management Systems

COURSE CODE: CSA05

CO5: *Implement database operations using query processing, optimization, and transaction management to ensure consistency and reliability. (BL3, BL4)*

Activity Tool : Pseudocode Writing Task (Algorithm Design) (Medium)

Assessment Tool Weightage: 35%

Dr

QUESTIONS			Total Marks: 50
Q.No	Question	Bloom's Level	Marks
1	Write pseudocode for the complete query processing pipeline: from SQL input through parsing, optimization, and execution to result output.	BL3 – Apply	5
2	Design a pseudocode algorithm for cost-based query optimization that estimates and compares I/O costs for different join orderings.	BL4 – Analyze	5
3	Write pseudocode for the Nested Loop Join algorithm and analyze its time complexity in terms of buffer pages and tuple counts.	BL3 – Apply	5
4	Write a pseudocode algorithm to detect deadlocks in a transaction system using a Wait-For Graph (WFG) cycle detection.	BL4 – Analyze	5
5	Design a pseudocode for the Basic Two-Phase Locking protocol including lock request, grant, wait, and release phases.	BL3 – Apply	5
6	Write pseudocode for timestamp-based concurrency control using Thomas' Write Rule for handling conflicting read/write operations.	BL4 – Analyze	5
7	Write a pseudocode algorithm for the ARIES (Algorithm for Recovery and Isolation Exploiting Semantics) recovery technique with Analysis, Redo, and Undo phases.	BL3 – Apply	5
8	Design pseudocode for Shadow Paging recovery: maintain current and shadow page tables and roll back on failure.	BL3 – Apply	5
9	Write a pseudocode for Immediate Update recovery using UNDO/REDO log processing after a system crash.	BL3 – Apply	5
10	Write pseudocode for a database connectivity manager that handles connection pooling, query execution, and exception rollback.	BL4 – Analyze	5

Code of Conduct:

I M.Devi Srilatha (192525236) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:**RUBRICS FOR CALCULATION:**

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	20%	Identifies all inputs, outputs, constraints accurately	Minor missing elements	Partial understanding	Incorrect interpretation
Algorithm Design	30%	Complete, correct, and logically sound solution	Minor logical gaps	Partially correct logic	Incorrect approach
Use of Control Structures	20%	Efficient and appropriate use of loops, conditions, functions/modules	Minor inefficiencies	Limited or basic usage	Incorrect or missing constructs
Pseudocode Structure	10%	Well-structured, clear, consistent format, easy to follow	Mostly clear with minor issues	Difficult to follow in parts	Poorly structured and unclear
Completeness	20%	Handles all cases; optimized approach	Handles most cases; reasonable efficiency	Handles basic cases only	Incomplete or inefficient

Q1. Complete Query Processing Pipeline

AIM

To design a query processing pipeline that converts an SQL query into an efficient execution plan and produces the required result.

PSEUDOCODE

ALGORITHM Query_Processing(SQL_Query)

INPUT: SQL_Query

OUTPUT: Query_Result

BEGIN

 Receive SQL_Query

 Parse SQL_Query

 IF syntax or semantic error exists THEN

 Display "Invalid Query"

 STOP

 END IF

 Convert SQL_Query into relational algebra

 Generate possible execution plans

 Estimate cost of each execution plan

 Select the plan with minimum cost

 Execute the selected execution plan

 Collect the output records

 Return Query_Result

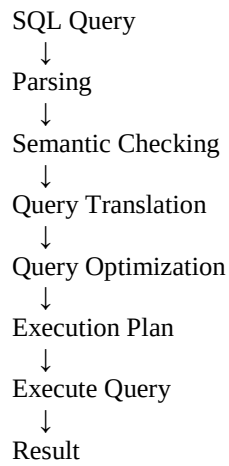
END

THEORY

Query processing is the process of converting an SQL query into a form that the database can execute efficiently. The query is first parsed to check its syntax and meaning. It is then converted into an internal representation. The optimizer generates possible execution plans

and selects an efficient plan based on cost. Finally, the selected plan is executed and the result is returned to the user.

FLOW



CONCLUSION

The query processing pipeline converts an SQL query into an efficient execution plan and executes it to obtain the required result with minimum cost.

Q2. Cost-Based Query Optimization

AIM

To design a cost-based query optimization algorithm that compares different join orders and selects the plan with the lowest I/O cost.

PSEUDOCODE

ALGORITHM Cost_Based_Optimization(Relations)

INPUT: Relations

OUTPUT: Best_Join_Order

BEGIN

 Generate possible join orderings

 Best_Cost $\leftarrow$ INFINITY

 Best_Join_Order $\leftarrow$ NULL

 FOR each Join_Order DO

 Estimate number of pages and tuples

 Estimate I/O cost for each join

 Total_Cost $\leftarrow$ Sum of all join costs

 IF Total_Cost < Best_Cost THEN

 Best_Cost $\leftarrow$ Total_Cost

 Best_Join_Order $\leftarrow$ Join_Order

 END IF

 END FOR

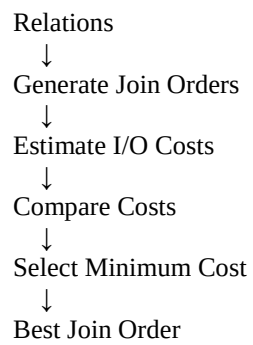
 Return Best_Join_Order

END

THEORY

Cost-based optimization evaluates different execution plans using estimated costs. The optimizer considers factors such as the number of tuples, number of pages, join order, and disk I/O. The plan with the lowest estimated cost is selected for execution.

FLOW



CONCLUSION

Cost-based optimization improves query performance by selecting the execution plan that requires the least estimated I/O cost.

Q3. Nested Loop Join

AIM

To design a Nested Loop Join algorithm for joining two relations and analyze its execution efficiency.

PSEUDOCODE

ALGORITHM Nested_Loop_Join(R, S)

INPUT: Relations R and S

OUTPUT: Joined_Relation

BEGIN

 Result $\leftarrow$ EMPTY

 FOR each tuple r in R DO

 FOR each tuple s in S DO

 IF Join_Condition(r, s) = TRUE THEN

 Add (r, s) to Result

 END IF

 END FOR

 END FOR

 Return Result

END

THEORY

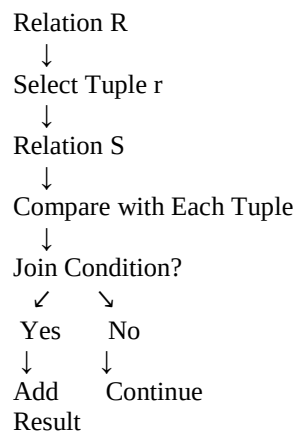
Nested Loop Join compares each tuple of the first relation with every tuple of the second relation. If the join condition is satisfied, the matching tuples are added to the result.

For relations containing $|R|$ and $|S|$ tuples, the basic time complexity is:

$O(|R| \times |S|)$

For page-oriented processing, the I/O cost depends on the number of buffer pages and pages in each relation.

FLOW



CONCLUSION

Nested Loop Join is simple and works with different join conditions, but its cost can become high when both relations contain a large number of tuples.

Q4. Deadlock Detection Using WFG

AIM

To detect deadlocks in a transaction system using a Wait-For Graph and cycle detection.

PSEUDOCODE

ALGORITHM Detect_Deadlock(WFG)

INPUT: Wait-For Graph

OUTPUT: Deadlock_Status

BEGIN

 Mark all transactions as UNVISITED

 FOR each transaction T in WFG DO

 IF T is UNVISITED THEN

 IF DFS_Cycle_Detection(T) = TRUE THEN

 Return "Deadlock Detected"

 END IF

 END IF

 END FOR

 Return "No Deadlock"

END

FUNCTION DFS_Cycle_Detection(T)

BEGIN

 Mark T as VISITING

 FOR each transaction U connected from T DO

 IF U is VISITING THEN

 Return TRUE

 END IF

 IF U is UNVISITED THEN

```

        IF DFS_Cycle_Detection(U) = TRUE THEN
            Return TRUE
        END IF

    END IF

END FOR

Mark T as VISITED

Return FALSE

END

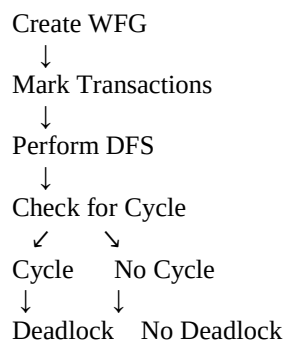
```

THEORY

A Wait-For Graph represents waiting relationships between transactions. Each transaction is represented as a node. If transaction T1 is waiting for T2, an edge is created from T1 to T2.

A cycle in the graph indicates a deadlock.

FLOW



CONCLUSION

Wait-For Graph cycle detection provides an effective method for identifying deadlocks between transactions.

Q5. Basic Two-Phase Locking

AIM

To design a Basic Two-Phase Locking algorithm that controls transaction access and maintains consistency.

PSEUDOCODE

ALGORITHM Two_Phase_Locking(Transaction T)

BEGIN

 Start Transaction T

 WHILE T requires a data item DO

 Request Lock

 IF Lock is available THEN

 Grant Lock

 ELSE

 Wait until Lock becomes available

 END IF

 END WHILE

 Enter Shrinking Phase

 WHILE T has locks DO

 Release Lock

 END WHILE

 Commit Transaction T

END

THEORY

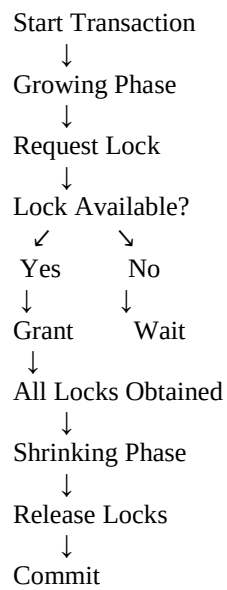
Two-Phase Locking divides transaction execution into two phases.

Growing Phase: The transaction can acquire locks but cannot release locks.

Shrinking Phase: The transaction can release locks but cannot acquire new locks.

This protocol helps ensure conflict-serializable schedules.

FLOW



CONCLUSION

Basic Two-Phase Locking controls concurrent transactions and helps maintain database consistency by separating lock acquisition and lock release.

Q6. Thomas' Write Rule

AIM

To design a timestamp-based concurrency control algorithm using Thomas' Write Rule to handle conflicting operations.

PSEUDOCODE

ALGORITHM Thomas_Write_Rule(T, Operation, X)

INPUT: Transaction T, Operation, Data Item X

BEGIN

IF Operation = READ(X) THEN

IF $TS(T) < Write_TS(X)$ THEN

Abort Transaction T

ELSE

Perform READ(X)

Read_TS(X) $\leftarrow$ MAX(Read_TS(X), TS(T))

END IF

ELSE IF Operation = WRITE(X) THEN

IF $TS(T) < Read_TS(X)$ THEN

Ignore WRITE(X)

ELSE IF $TS(T) < Write_TS(X)$ THEN

Ignore WRITE(X)

ELSE

Perform WRITE(X)

Write_TS(X) $\leftarrow$ TS(T)

END IF

END IF

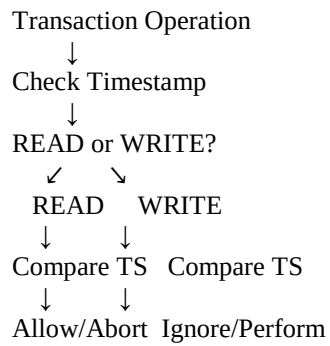
END

THEORY

Timestamp ordering assigns a unique timestamp to each transaction. Read_TS and Write_TS are maintained for each data item.

Thomas' Write Rule allows certain outdated write operations to be ignored instead of aborting the transaction. This can improve concurrency and reduce unnecessary transaction failures.

FLOW



CONCLUSION

Thomas' Write Rule improves concurrency by safely ignoring obsolete write operations instead of unnecessarily aborting transactions.

Q7. ARIES Recovery

AIM

To design an ARIES recovery algorithm that restores database consistency after a system crash.

PSEUDOCODE

ALGORITHM ARIES_Recovery(Log)

INPUT: Database Log

OUTPUT: Recovered Database

BEGIN

 Analysis Phase

 Read Log from Last Checkpoint

 Identify Active Transactions

 Identify Dirty Pages

 Determine REDO Starting Point

 Redo Phase

 Scan Log Forward

 FOR each Log Record DO

 IF Operation must be repeated THEN

 REDO Operation

 END IF

 END FOR

 Undo Phase

 Identify Uncommitted Transactions

 Scan their Log Records Backward

 FOR each Update Operation DO

 UNDO Operation

 Write Compensation Log Record

END FOR

Mark Uncommitted Transactions as Aborted

Complete Recovery

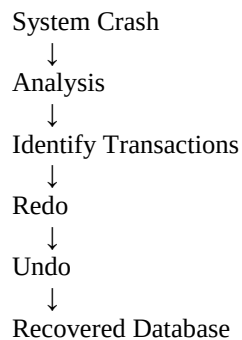
END

THEORY

ARIES is a database recovery technique that uses logging to recover from system failures. It contains three phases: **Analysis, Redo, and Undo.**

Analysis identifies the state of transactions and pages. Redo repeats required operations. Undo reverses changes made by incomplete transactions.

FLOW



CONCLUSION

ARIES provides reliable database recovery by using Analysis, Redo, and Undo phases to restore the database to a consistent state.

Q8. Shadow Paging

AIM

To design a Shadow Paging recovery algorithm that maintains current and shadow page tables and restores the database after failure.

PSEUDOCODE

ALGORITHM Shadow_Paging_Recovery()

BEGIN

 Create Shadow_Page_Table

 Current_Page_Table $\leftarrow$ Copy(Shadow_Page_Table)

 Start Transaction

 WHILE Transaction performs updates DO

 Create New Page

 Write Updated Data to New Page

 Update Current_Page_Table

 END WHILE

 IF Transaction Commits Successfully THEN

 Make Current_Page_Table Permanent

 Delete Old Shadow_Page_Table

 ELSE IF System Failure Occurs THEN

 Discard Current_Page_Table

 Restore Shadow_Page_Table

 END IF

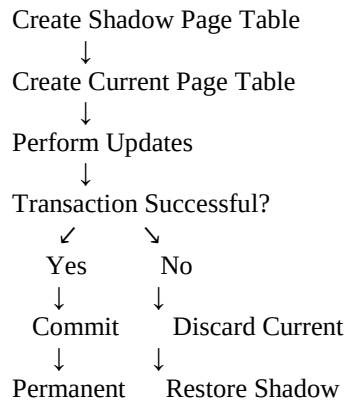
END

THEORY

Shadow paging maintains two page tables. The shadow page table contains the stable database mapping, while the current page table contains changes made during the transaction.

If the transaction commits, the current page table becomes permanent. If a failure occurs, the current page table is discarded and the shadow page table is used to restore the previous state.

FLOW



CONCLUSION

Shadow paging provides simple recovery by preserving the original page table until the transaction successfully commits.

Q9. Immediate Update Recovery

AIM

To design an Immediate Update recovery algorithm using UNDO and REDO operations after a system crash.

PSEUDOCODE

ALGORITHM Immediate_Update_Recovery(Log)

INPUT: Database Log

OUTPUT: Recovered Database

BEGIN

 Identify Committed Transactions

 Identify Uncommitted Transactions

 REDO Phase

 FOR each update of committed transactions DO

 Write New_Value to Database

 END FOR

 UNDO Phase

 FOR each update of uncommitted transactions

 in reverse order DO

 Write Old_Value to Database

 END FOR

 Complete Recovery

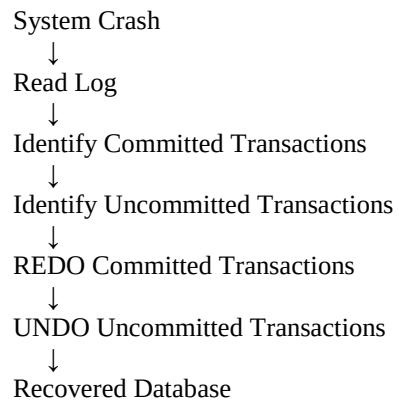
END

THEORY

In immediate update recovery, database changes may be written before a transaction commits. Therefore, the log stores old and new values.

Committed transactions are processed using **REDO**, while incomplete transactions are processed using **UNDO**.

FLOW



CONCLUSION

Immediate Update recovery maintains consistency by reapplying committed changes and reversing incomplete changes after a crash.

Q10. Database Connectivity Manager

AIM

To design a database connectivity manager that handles connection pooling, query execution, transactions, and exception rollback.

PSEUDOCODE

ALGORITHM Database_Connectivity_Manager(Query)

INPUT: Database Query

OUTPUT: Query_Result

BEGIN

 Initialize Connection_Pool

 Request Connection from Connection_Pool

 IF Connection is unavailable THEN

 Wait until Connection becomes available

 END IF

 Get Connection

 BEGIN TRANSACTION

 TRY

 Execute Query

 IF Query executes successfully THEN

 COMMIT TRANSACTION

 Store Query_Result

 ELSE

 RAISE Exception

 END IF

 CATCH Exception

 ROLLBACK TRANSACTION

 Display Error Message

 FINALLY

 Return Connection to Connection_Pool

 END TRY

 Return Query_Result

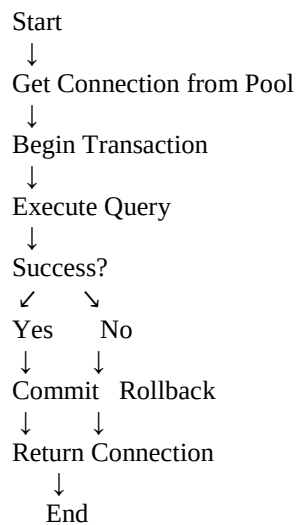
END

THEORY

A database connectivity manager controls communication between an application and the database. Connection pooling allows existing connections to be reused instead of creating a new connection for every query.

The query is executed inside a transaction. If execution succeeds, the transaction is committed. If an exception occurs, the transaction is rolled back and the connection is returned to the pool.

FLOW



CONCLUSION

A database connectivity manager improves performance through connection pooling and maintains reliability through transaction, commit, rollback, and exception handling.